

ECOM EXPERTS

WithdrawalKit EU

Manual de usuario e instalacion

desistimiento-shopify 1.0.0 (app de Shopify) · 04/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

3.1 Instalación del servicio (una sola vez, la hace el desarrollador)

3.2 Instalación en una tienda (la hace el comerciante)

3.3 Qué pasa al desinstalar

4. Configuración

Pestaña General

Pestaña Estados y plazo

Pestaña Productos excluidos

Pestaña B2B / Profesionales

Pestaña Emails

5. Cómo se usa, día a día

Panel de inicio

Cuando entra una solicitud

Tramitar una solicitud

Plan y facturación

6. Qué ve el cliente en la tienda

7. Preguntas frecuentes y problemas

8. Seguridad

Lo que ya estaba bien

Lo que se ha arreglado en esta revisión

Lo que queda pendiente

9. Ficha técnica

Tablas (Prisma, MariaDB)

Rutas de la aplicación

Webhooks

Ficheros de configuración y despliegue

Cron y permisos

Registros

1. Qué hace y para quién

Cualquier tienda online que venda a consumidores de la Unión Europea está obligada a permitir que el comprador se eche atrás. La ley española (Real Decreto Legislativo 1/2007, el TRLGDCU, artículos 102 a 108) y la Directiva 2011/83/UE le dan al cliente catorce días naturales desde que recibe el producto para desistir del contrato sin dar explicaciones y sin penalización. El comerciante tiene que devolverle el dinero, incluidos los gastos de envío estándar, en otros catorce días como máximo. Si la tienda nunca informó de ese derecho, el plazo del cliente no es de catorce días: se estira hasta doce meses más (artículo 105).

En la práctica, casi ninguna tienda de Shopify tiene esto resuelto. El cliente escribe un correo, el comerciante lo pierde entre otros cincuenta, no queda constancia de cuándo se pidió, y cuando llega una reclamación de consumo no hay forma de demostrar en qué fecha se comunicó el desistimiento ni qué se contestó. A eso se suma que desde el 19 de junio de 2026 la Directiva UE 2023/2673 exige un enlace visible y permanente al formulario de desistimiento en la propia tienda, no enterrado en una página de condiciones.

WithdrawalKit EU cubre las dos mitades del problema. Por el lado del cliente pone el formulario oficial del Anexo B del TRLGDCU en la tienda: un enlace en el pie de página, un icono flotante y un botón dentro del área de «Mis pedidos» de la cuenta del cliente. El comprador rellena sus datos, marca los productos de los que desiste, firma la declaración y envía. Por el lado del comerciante, cada solicitud entra en un panel dentro del propio administrador de Shopify, con su estado (recibida, en revisión, aceptada, producto recibido, reembolsada o rechazada), su historial de cambios con fecha y autor, y los importes que se van a devolver desglosados entre producto, envío y depreciación.

No es una app de devoluciones al uso. Una devolución comercial es una política voluntaria de la tienda; el desistimiento es un derecho legal con plazos, formato y consecuencias fijadas por ley. Por eso la app trae los textos legales por defecto ya redactados sobre el articulado, distingue entre compra de consumidor y compra B2B —los profesionales no tienen este derecho— y permite excluir los productos que la propia ley deja fuera (artículo 103: bienes personalizados, precintados por higiene, contenido digital ya descargado y demás).

2. Compatibilidad

- **Plataforma:** Shopify. Aplicación embebida en el administrador, distribución App Store.
- **Versión de la API de Shopify:** `2026-01` (constante `ApiVersion.January25` en `app/shopify.server.ts`; las extensiones declaran `api_version = "2026-01"`).
- **Node.js:** 18.20, 20.10 o superior a 21 (campo `engines` de `package.json`). En producción se usa Node 22 LTS.
- **Base de datos:** MariaDB o MySQL, a través de Prisma 6.
- **Framework:** Remix 2.16 sobre React 18, con Polaris 13 y App Bridge 4.

- **Permisos que pide la app** (`shopify.app.toml`): `read_orders` , `read_customers` , `read_products` , `read_themes` , `read_locales` . Todos de lectura: la app nunca modifica el catálogo ni los pedidos.
- **Idiomas**: la interfaz del comerciante y el formulario público están en castellano. La estructura de datos guarda los textos legales por idioma (`{"es": "...", "en": "..."}`) y los planes Standard y Pro anuncian siete idiomas, pero **hoy solo la clave es se escribe y se lee**. Ver el apartado 7.
- **Multitienda**: sí, por diseño. Cada tienda de Shopify que instala la app es una fila en la tabla `Shop` y toda la configuración y todas las solicitudes cuelgan de ella.
- **Extensiones que se despliegan con la app**:
 - *Theme App Extension* con dos bloques: icono flotante y enlace de pie de página.
 - *Customer Account UI Extension* con tres puntos de entrada: elemento de menú en el pedido, pantalla de acción, y bloque en el listado de pedidos.

3. Instalación

La app no se instala como un módulo que se sube a la tienda: es un servicio que corre en un servidor propio y al que la tienda se conecta. Hay, por tanto, dos instalaciones distintas.

3.1 Instalación del servicio (una sola vez, la hace el desarrollador)

1. Subdominio propio con certificado (en este despliegue, `https://desistimiento.gmartos.es`) y base de datos MariaDB vacía.
2. Subir el código al servidor **sin** `node_modules` , `build` , `.env` ni `storage/logs` : la lista está en `.ftppignore` .
3. Crear el `.env` en el servidor a partir de `.env.example` , rellenando la clave y el secreto de la app de Shopify Partners, la URL pública, la cadena de conexión a la base de datos y un `SESSION_SECRET` nuevo (`openssl rand -base64 48`). **La contraseña de la base de datos va URL-encoded**: un `}` se escribe `%7D` y un `$` se escribe `%24` , o Prisma no sabe leer la cadena.
4. Ejecutar el despliegue: `bash deploy/setup.sh` la primera vez (instala dependencias, genera el cliente de Prisma, crea las tablas, compila y arranca PM2) o `bash deploy/deploy.sh` en las siguientes.
5. Poner el `snippet` de nginx de `deploy/nginx-vhost.conf` como configuración personalizada del vhost, para HTTP y para HTTPS. Hace de proxy hacia el puerto 3001 y bloquea el acceso directo a `.env` , `.git` , `deploy/` , `prisma/` , `app/` y `storage/` .
6. Sincronizar la configuración con Shopify Partners: `npx shopify app deploy` .

El detalle paso a paso, con las capturas del panel de HestiaCP, está en `INSTALL.md` , en esta misma carpeta.

3.2 Instalación en una tienda (la hace el comerciante)

Se instala desde la ficha de la app. Al aceptar los permisos, Shopify redirige al administrador de la tienda y la app crea automáticamente, en su propia base de datos:

- Una fila en `Shop` con el dominio de la tienda, la fecha de instalación y un periodo de prueba de catorce días.

- Una fila en `ShopConfig` con la configuración por defecto, incluidos los tres textos legales ya redactados en castellano.

No toca nada de la tienda: ni tablas, ni metacampos, ni plantillas. Lo único que hay que hacer a mano después es **activar el enlace visible en el tema**, que es justamente lo que exige la normativa; el panel de inicio lo recuerda con un aviso rojo y dos botones que abren el editor de temas en el sitio correcto.

3.3 Qué pasa al desinstalar

Shopify envía el webhook `app/uninstalled`. La app marca la tienda con `uninstalledAt` y borra sus sesiones, pero **conserva las solicitudes**: son documentación legal de operaciones ya cerradas y borrarlas dejaría al comerciante sin prueba ante una reclamación. El borrado real solo ocurre con el webhook `shop/redact` que Shopify manda cuarenta y ocho horas después de una desinstalación definitiva; entonces sí se eliminan la configuración, las solicitudes, los registros y la propia tienda.

4. Configuración

La configuración vive en **Configuración**, dentro del menú lateral de la app, repartida en cinco pestañas. Todas guardan con su propio botón «Guardar cambios» y confirman con una franja verde.

No hay capturas de pantalla en este manual. Este proyecto es una app de Shopify: no se instala en las tiendas PrestaShop locales y el capturador de pantallas del flujo habitual no le sirve. Las pantallas se documentan por escrito.

Pestaña General

- **Activar botón de desistimiento en la tienda.** Es el interruptor maestro. Desactivado, la app sigue instalada y los datos siguen ahí, pero deja de ofrecerse el formulario. Déjalo activado salvo que estés preparando la tienda antes de abrirla.
- **Mostrar icono flotante.** Controla el círculo naranja que queda fijo en una esquina de la tienda. Es la forma más directa de cumplir con el requisito de enlace visible. Ojo: este interruptor es la intención del comerciante, pero **el icono solo aparece de verdad si además se activa el bloque en el editor de temas** (botón «Activar icono flotante en el tema» de la pantalla de inicio).
- **Mostrar solo en países UE/EEA.** Si vendes también fuera de Europa, evita ofrecer un derecho que allí no aplica. La comprobación se hace en la plantilla del tema contra el país de la tienda.
- **Activar logs de depuración.** Solo para diagnosticar un problema concreto con soporte. Déjalo apagado en el día a día.
- **¿Quién paga la devolución?** Determina qué se le dice al cliente sobre el coste del transporte de vuelta. La ley permite repercutírselo **siempre que se le haya informado antes de la compra**; si no se le informó, lo paga el comerciante. Elige «El cliente» solo si tus condiciones de venta lo dicen expresamente.
- **Datos del empresario.** Nombre o razón social, dirección postal, teléfono y correo. No es decorativo: el Anexo B exige que el consumidor sepa a quién dirige su declaración, y estos datos son los que aparecen

en el formulario público y en el resguardo.

- **Texto legal del desistimiento.** Viene relleno con la información obligatoria redactada sobre los artículos 102 a 108. Si lo dejas vacío se repone el texto por defecto. Cámbialo solo si tu asesoría te da una redacción propia.
- **Apariencia del botón.** Forma (redondeado, cápsula o cuadrado) y dos colores en hexadecimal. **Los colores se validan:** si escribes algo que no sea un # seguido de tres o seis dígitos hexadecimales, se usa el color por defecto y no se rompe la página (ver apartado 8).

Pestaña Estados y plazo

- **Días de plazo legal.** Catorce por defecto. Hay atajos para 14, 30, 60 y 365. Los 365 son el caso del artículo 105: si no informaste del derecho de desistimiento, el plazo del consumidor se amplía. El valor se limita entre 7 y 365 en el propio servidor.
- **Desde qué estado empieza a contar.** Esta es la opción que más se malinterpreta. La ley cuenta desde que el cliente **recibe físicamente** el producto, no desde que paga. Por eso lo correcto en una tienda de producto físico es «entregado» o, si tu transportista no confirma entregas, «preparado/enviado». «Creado» solo tiene sentido en servicios o contenido digital. Cuidado con un detalle: si eliges «entregado» y un pedido nunca llega a marcarse como entregado, para ese pedido el plazo **no ha empezado a contar** y sigue abierto indefinidamente.

Pestaña Productos excluidos

Cuatro listas —productos, colecciones, fabricantes y etiquetas— más el texto legal de las exclusiones. Sirven para reflejar lo que el artículo 103 deja fuera del derecho: lo hecho a medida, lo precintado por higiene una vez abierto, lo perecedero, el contenido digital ya empezado. Se escriben una a una y se añaden con el botón; se guardan como lista separada por comas. Las comparaciones de fabricante y etiqueta no distinguen mayúsculas de minúsculas; **la de producto y colección sí**, y se hace por identificador exacto.

Pestaña B2B / Profesionales

El derecho de desistimiento protege al consumidor final, no a quien compra en el ejercicio de su actividad. Aquí se decide qué hacer con los pedidos de empresa:

- **Bloquear o permitir.** Bloquear es lo ajustado a la ley; permitir es una cortesía comercial perfectamente válida.
- **Detección por campo «empresa», por NIF/CIF/VAT y por etiquetas de cliente.** La detección por identificación fiscal comprueba que haya al menos cinco caracteres alfanuméricos: **valida que exista, no que sea un NIF real.** Si tu tienda pide el NIF a todos los clientes, incluidos los particulares, esta casilla marcará como B2B a medio catálogo de clientes; en ese caso desactívala y usa etiquetas.

Pestaña Emails

Shopify no deja que una app externa mande correos en nombre de la tienda. Lo que hace la app es **darte el fragmento de código Liquid ya montado** para que lo pegues, una sola vez, en las notificaciones nativas que elijas (confirmación de pedido, factura, confirmación de envío, actualización de envío, entrega confirmada). Eliges el texto, si va antes o después del contenido, copias el fragmento y lo pegas en Shopify, en Configuración → Notificaciones.

5. Cómo se usa, día a día

Panel de inicio

Nada más entrar hay cuatro contadores —total de solicitudes, pendientes, reembolsadas y rechazadas— y, encima, el aviso rojo del enlace obligatorio con los dos botones que abren el editor de temas. Ese aviso no desaparece solo: está pensado para que no se olvide.

Cuando entra una solicitud

El cliente rellena el formulario en la tienda y la solicitud aparece en **Solicitudes** con estado «Recibida». El listado enseña el número de pedido, el cliente con su correo, la fecha, el estado, si es B2B, cuántas líneas lleva y el importe. Se puede filtrar por estado y buscar por pedido, nombre o correo.

Tramitar una solicitud

Al abrirla se ven, en este orden: los datos del cliente y su motivo, las observaciones que escribió, los productos que reclama, el bloque de cambio de estado, el de importes y el historial.

1. **Cambiar el estado** conforme avanza el caso, con una nota opcional. Cada cambio queda registrado en el historial con la fecha y el correo del empleado que lo hizo. Ese historial es la prueba documental ante una reclamación de consumo: úsalo.
2. **Anotar los importes**: producto, envío estándar y depreciación. La depreciación es lo que se descuenta cuando el cliente ha usado el artículo más allá de lo necesario para probarlo, y la ley obliga a justificarla, así que rellena siempre la nota. El total se calcula solo, en pantalla.
3. **Ejecutar el reembolso**. Este es el punto que más confusión genera: **la app no mueve dinero**. El botón «Abrir reembolso en Shopify» lleva a la pantalla de reembolso del pedido en el administrador de Shopify, y es ahí donde se devuelve. Los importes de la app son el registro para auditoría, no la orden de pago. Después, marca la solicitud como «Reembolsada» para que quede constancia de la fecha.

Plan y facturación

Tres planes recurrentes —Starter 9,99 €, Standard 19,99 € y Pro 39,99 €, todos al mes y con catorce días de prueba— contratados con el sistema de facturación de Shopify. **Aviso importante para el desarrollador:** hoy las suscripciones se crean siempre en modo de prueba y no llegan a cobrarse. Está explicado en el apartado 7.

6. Qué ve el cliente en la tienda

Tres puertas de entrada, todas al mismo formulario:

- **El icono flotante**, un círculo fijo en una esquina de todas las páginas.
- **El enlace en el pie de página**, con el texto que elija el comerciante.

- **El botón «Desistir del contrato»** dentro de «Mis pedidos» de la cuenta del cliente, más un aviso informativo en el listado de pedidos.

Cualquiera de ellos abre <https://<la-tienda>/apps/desistimiento>, una página servida por la app pero **bajo el dominio de la tienda**, con los colores que haya elegido el comerciante. El recorrido tiene tres pasos, indicados arriba con una barra de progreso:

1. **Localizar el pedido** con su número y el correo de la compra. Debajo se muestran ya el texto legal del desistimiento, la lista de exclusiones y los datos del empresario, que es la información que la normativa obliga a poner delante del consumidor.
2. **Rellenar el Anexo B:** nombre, teléfono opcional, correo de contacto, motivo (con la aclaración de que legalmente no está obligado a darlo, que se pide solo con fines estadísticos), observaciones, los productos de los que desiste y la casilla de declaración expresa citando los artículos.
3. **Confirmación** en pantalla, con una referencia interna, el resumen de lo enviado y los pasos siguientes.

Si vuelve a entrar con el mismo pedido y correo, se le enseña la solicitud que ya tenía con su estado actual, en lugar de crear otra.

7. Preguntas frecuentes y problemas

He activado «Mostrar icono flotante» y no sale nada. Son dos interruptores distintos y hay que dar los dos. El de la app dice que quieres el icono; el del **editor de temas** lo pinta. Usa el botón «Activar icono flotante en el tema» de la pantalla de inicio, que abre el editor justo donde hay que activarlo, y guarda allí.

Un cliente dice que ha enviado el formulario pero no aparece en Solicitudes. Comprueba que buscas en el filtro correcto y sin texto en la caja de búsqueda. Si aun así no está, mira que el interruptor maestro de la pestaña General esté activado.

¿La app devuelve el dinero automáticamente? No, y es deliberado. El reembolso se hace en Shopify, en la pantalla del pedido. La app guarda los importes y las fechas para tener la trazabilidad, pero no tiene permiso de escritura sobre los pedidos: los cinco permisos que pide son de solo lectura.

El plazo de un pedido no avanza nunca. Pasa cuando en «Estados y plazo» está elegido «entregado» y el pedido nunca se marcó como entregado en Shopify. El plazo legal empieza con la recepción; si esa recepción no consta, para el sistema no ha empezado. Si tu transportista no confirma entregas, usa «preparado/enviado».

Todos mis clientes salen marcados como B2B. Tienes activada la detección por NIF y tu tienda pide el NIF también a particulares. La comprobación mira que el campo esté relleno, no que sea una empresa. Desactívala y usa etiquetas de cliente para marcar solo las cuentas profesionales.

Puse 365 días sin querer y ahora todos los pedidos están en plazo. Vuelve a la pestaña «Estados y plazo» y pon 14. Los 365 son para el supuesto del artículo 105, cuando no se informó del derecho; no es un valor «generoso», es un caso legal concreto.

El formulario está en castellano, ¿y mis clientes extranjeros? Ahora mismo el formulario público y la interfaz están en castellano. La base de datos ya guarda los textos por idioma y los planes anuncian siete,

pero solo la clave `es` está conectada. Es una funcionalidad pendiente, no un fallo de configuración: no busques dónde cambiarlo, todavía no existe.

El contador del icono flotante siempre está a cero. Es un fallo conocido y pendiente: el punto `/apps/desistimiento/count` devuelve la página HTML del formulario en vez de un número, así que el JavaScript del tema no puede leerlo y deja el globo escondido. No afecta al funcionamiento del formulario. Ver el apartado 8.

¿Se puede probar la app en local? El proyecto trae `setup-local.ps1` para montar el entorno de desarrollo en Windows, pero una app de Shopify necesita una URL pública y una tienda de desarrollo: no se prueba abriendo un fichero.

8. Seguridad

Se ha revisado el código entero de la aplicación: el fichero de arranque, las veinte rutas de Remix, los dos servicios, el manejador del proxy público, las cinco extensiones, el esquema de Prisma y los cuatro guiones de despliegue. Se ha comprobado en concreto lo propio de una app de Shopify sobre Node y Remix: la verificación HMAC de los webhooks, la validación de la sesión de App Bridge, los secretos en el código, la inyección en consultas, las cabeceras CORS y de caché, las dependencias con vulnerabilidades conocidas y el registro de datos personales en los ficheros de registro.

Lo que ya estaba bien

- **Los webhooks se autentican de verdad.** `authenticate.webhook()` verifica la firma HMAC antes de dar por buena la carga útil, y hay una tabla `WebhookSeen` que descarta los reenvíos duplicados de Shopify.
- **El formulario público está firmado.** Las cinco rutas del proxy pasan por `authenticate.public.appProxy()`, que comprueba la firma que Shopify añade a la petición: no se puede llamar al formulario desde fuera de la tienda.
- **Las cuatro tareas de cumplimiento del RGPD están implementadas** y borran o anonimizan de verdad (`customers/redact`, `shop/redact`, `customers/data_request`, `app/uninstalled`).
- **No hay SQL escrito a mano.** Todo pasa por Prisma con parámetros; no existe una sola consulta concatenada, ni un `ORDER BY` construido con datos del usuario.
- **No hay `eval`, ni `exec`, ni `innerHTML`, ni `dangerouslySetInnerHTML`** en ninguna parte. El HTML del formulario público se monta escapando todas las variables.
- **La configuración del servidor web ya bloqueaba** el acceso directo a `.env`, `.git`, `deploy/`, `prisma/`, `app/`, `node_modules/` y `storage/`.
- **Nadie define cabeceras CORS**, así que no hay ningún origen externo autorizado a llamar a la app. Correcto para lo que hace.

Lo que se ha arreglado en esta revisión

1. Fuga de datos entre tiendas distintas en el formulario público. `app/lib/proxy-handler.server.ts` buscaba las solicitudes por correo del cliente y número de pedido **sin filtrar por tienda**. Dos comercios distintos con el mismo número de pedido —cosa nada rara, todos empiezan por `#1001`— podían ver el uno

los datos del otro: nombre, estado de la solicitud y fecha. Es la falla más grave encontrada. Se ha añadido `shopId` a las dos consultas.

2. Un comerciante podía escribir en las solicitudes de otro. `setRefundAmount()`, en `app/services/desist.server.ts`, buscaba la tienda y luego actualizaba la solicitud **por su identificador a secas**, sin comprobar que fuera suya. El post-mortem del proyecto ya avisaba de este patrón en `getRequest()`, que sí se corrigió en su momento; esta función se quedó atrás. Ahora es un `updateMany` filtrado por `shopId` que además falla si no ha tocado ninguna fila, y los importes se acotan a valores positivos.

3. Inyección de código en la página que ve el cliente, a través de los colores. Los dos colores de marca se metían tal cual dentro de un bloque `<style>` de la página del formulario. Un valor como `red}</style><script>...` habría cerrado la hoja de estilos e inyectado JavaScript **en una página servida bajo el dominio de la tienda**. Se ha añadido `safeColor()`: solo se acepta un hexadecimal de tres o seis dígitos y cualquier otra cosa cae al color por defecto.

4. Credenciales de sesión escritas en los ficheros de registro. `app/routes/auth.$.tsx` tenía un bloque de depuración `[AUTH-DEBUG]` que volcaba la cabecera `Authorization` al registro. Los ficheros `storage/logs/pm2-out.log` que venían en el proyecto **contenían treinta y seis tokens de sesión JWT reales**. Se ha eliminado el bloque entero, se han borrado los dos ficheros de registro y se ha dejado un `.gitignore` en `storage/logs/` para que no vuelvan a viajar en el paquete.

5. Datos personales en el registro del RGPD. El manejador de `customers/data_request` escribía el identificador del cliente en el registro. Ahora solo anota el número de solicitudes encontradas.

6. Secretos de producción dentro del paquete. El `.env` que venía en la carpeta del proyecto tenía valores **reales**: el secreto de la app de Shopify (`shps_...`), el `SESSION_SECRET` y la contraseña de MariaDB. Se ha sacado de la carpeta y se ha dejado en su sitio un `.env.example` con marcadores. Además, la contraseña de la base de datos estaba escrita en claro **dos veces más**: en `deploy/setup.sh` y en `deploy/setup.php`. En los dos casos ahora se lee del entorno o del propio `.env` del servidor.

Acción pendiente para el usuario, importante: esos tres secretos han estado en una carpeta que se distribuye. Hay que **rotarlos**: regenerar el secreto de la app en Shopify Partners, generar un `SESSION_SECRET` nuevo con `openssl rand -base64 48` y cambiar la contraseña de MariaDB. La copia con los valores antiguos, por si hace falta consultarlos, está fuera del paquete, en `modulos IA/utiles/desistimiento-shopify/env-produccion-NO-DISTRIBUIR.txt`.

7. Ejecutor de despliegue accesible desde el navegador. `deploy/setup.php` lanza órdenes en el servidor — `rm -rf`, `npm install`, `prisma db push --accept-data-loss`, `DELETE FROM` de varias tablas— y estaba protegido solo por un testigo escrito en el propio fichero con la palabra `CAMBIAME` dentro. Cualquiera que diera con la URL y leyera el código habría podido reinstalar y vaciar la base de datos. Ahora el testigo se lee del entorno del servidor, el fichero **se niega a arrancar** si no está definido o tiene menos de veinticuatro caracteres, la comparación usa `hash_equals` para no filtrarlo por el tiempo de respuesta, hay una espera de medio segundo en cada intento fallido y la acción se comprueba contra la lista de las siete permitidas. La cabecera del fichero recuerda que hay que borrarlo del servidor al terminar.

8. Ausencia de cabeceras anti-caché en el formulario público. Las páginas del proxy enseñan el estado de la solicitud de un cliente concreto y se sirven por detrás de la red de distribución de Shopify. Sin cabeceras, la respuesta de un cliente podía acabar entregada a otro. Se han añadido las tres — `Cache-Control`, `Pragma` y `Expires` — a las cinco respuestas, sin tocar el `Content-Type`.

9. Entrada del formulario público sin límites ni validación. Cualquier visitante de la tienda llega al formulario. No había tope de longitud en ningún campo, el motivo no se comprobaba contra la lista de motivos válidos, el correo no se validaba y cada envío repetido creaba una fila nueva. Ahora todos los campos se recortan (4.000 caracteres el listado de productos, 2.000 las observaciones, 190 los correos), el motivo se valida contra la lista cerrada, el correo tiene que tener forma de correo, faltan datos obligatorios devuelve un 400, y un segundo envío del mismo pedido y correo muestra la solicitud que ya existe en vez de duplicarla.

10. Estado de solicitud y plan de facturación sin lista blanca. El estado llegaba de un desplegable y se guardaba tal cual: un valor inventado se habría almacenado y habría dejado la ficha de la solicitud rota, porque la pantalla busca el estado en una tabla fija. Igual con el nombre del plan, que iba de un campo oculto directo a la API de facturación. Los dos se comprueban ahora contra su lista cerrada.

11. Compilado antiguo dentro del paquete. La carpeta `build/` traía la versión compilada **anterior a estos arreglos**. Se ha eliminado para que no se despliegue por error; los dos guiones de despliegue la regeneran con `npm run build`.

Lo que queda pendiente

A. El formulario no comprueba que el pedido exista ni que sea del cliente. Es la limitación de fondo. La app pide permiso `read_orders` pero **no llama ni una sola vez a la API de Shopify**: la búsqueda del pedido solo mira su propia tabla. En la práctica, cualquier visitante puede presentar una solicitud de desistimiento con un número de pedido y un correo inventados, y le entrará al comerciante como legítima. No se ha tocado porque arreglarlo significa integrar la API de pedidos de Shopify y rehacer el paso de búsqueda: es funcionalidad nueva, no un parche. Mientras tanto, el comerciante debe contrastar cada solicitud contra el pedido real antes de aceptarla. Del mismo modo, tampoco se aplican en el formulario público el plazo configurado, las exclusiones ni las reglas B2B: las funciones que los calculan (`deadlineFor`, `isLineExcluded`, `isB2B`) están escritas y probadas, pero nadie las llama todavía.

B. Las suscripciones nunca se cobran. En `app/routes/app.billing.tsx`, la línea `isTest:` `!session.shop.endsWith(".myshopify.com") ? false : true` deja el modo de prueba activado para **todas** las tiendas, porque el dominio interno de toda tienda de Shopify termina en `.myshopify.com`. Los tres planes se contratan sin cobrar. No se ha cambiado porque afecta al dinero y a las pruebas en curso: es una decisión del usuario, no una corrección de seguridad.

C. El contador del icono flotante no funciona. `app/routes/count.tsx` reexporta el manejador del formulario, así que `/apps/desistimiento/count` devuelve HTML donde el JavaScript del tema espera un JSON con un número. El error se traga silenciosamente y el globo no aparece nunca.

D. Cuarenta y seis paquetes con vulnerabilidades conocidas, según `npm audit` sobre `package-lock.json`: una crítica, treinta y cuatro altas y once moderadas.

Conviene separarlas, porque no todas pesan igual. Las que afectan al **servidor en producción** son estas:

Paquete	Gravedad	Qué es
<code>react-router-dom</code>	moderada	Redirección abierta que deriva en XSS
<code>react-router</code>	moderada	Inyección de constructor arbitrario en <code>deserializeErrors()</code>
<code>turbo-stream</code>	alta	Denegación de servicio por entrada reflejada en la carga única
<code>undici</code>	moderada	Inyección CRLF, desincronización de respuestas, inyección de cookies
<code>qs</code>	moderada	Denegación de servicio por <code>isBuffer</code> controlable
<code>lodash</code>	alta	Inyección de código vía <code>_.template</code> ; contaminación de prototipo en <code>_.unset</code>
<code>body-parser</code>	baja	Denegación de servicio con un <code>limit</code> inválido

Las cuatro primeras llegan arrastradas por `@remix-run/server-runtime`, del que cuelgan `@remix-run/node`, `@remix-run/express`, `@remix-run/serve` y el propio `@shopify/shopify-app-remix`. El resto —la crítica de `tar`, y `vite`, `esbuild`, `postcss`, `browserslist`, `js-yaml`, `shell-quote`, `nanoid`, `immutable`, `toml` — son herramientas de compilación: solo cuentan en la máquina donde se construye la app, no en el servidor que la sirve. Sigue mereciendo la pena actualizarlas, pero no son la urgencia.

No se han actualizado en esta revisión, y a propósito. Subir la versión mayor de Remix o del SDK de Shopify es una migración, no un parche, y hacerla sin poder arrancar el servidor ni ejecutar una sola prueba —este repaso se ha hecho leyendo el código, sin `node_modules` instalados— sería cambiar un riesgo conocido por uno desconocido. Es lo primero que hay que abordar en la siguiente sesión, con el entorno levantado: `npm audit fix` para lo que no rompe nada, y evaluar a mano el salto de Remix y del SDK.

E. Los ficheros de registro no rotan. PM2 escribe en `storage/logs/` sin límite de tamaño. Conviene configurar `pm2-logrotate` en el servidor.

F. No hay límite de peticiones al formulario público. El control de duplicados que se ha añadido evita las filas repetidas del mismo pedido, pero nada impide a un guion enviar miles de solicitudes con pedidos distintos. Lo natural es resolverlo en nginx con `limit_req_zone` sobre la ruta del proxy.

9. Ficha técnica

Tablas (Prisma, MariaDB)

Tabla	Para qué
<code>Session</code>	Sesiones de Shopify, gestionadas por el propio SDK. Con testigos de refresco.
<code>Shop</code>	Una fila por tienda instalada: dominio, fechas de instalación y desinstalación, plan, periodo de prueba y contadores.
<code>ShopConfig</code>	Toda la configuración de una tienda. Los textos legales, los datos del empresario y las listas de exclusión se guardan como JSON por idioma.
<code>DesistRequest</code>	La solicitud: pedido, cliente, estado, motivo, si es B2B y los tres importes en <code>Decimal(20,6)</code> .
<code>DesistRequestDetail</code>	Las líneas de producto de cada solicitud.
<code>DesistLog</code>	Auditoría: cada cambio de estado con origen, destino, nota, tipo de actor y fecha.
<code>WebhookSeen</code>	Identificadores de webhook ya procesados, para no repetir el trabajo con los reenvíos.

Rutas de la aplicación

Ruta	Qué hace
<code>/</code>	Triple función: si lleva <code>signature</code> es el proxy, si lleva <code>shop</code> y <code>host</code> redirige al panel, si no muestra la portada pública.
<code>/app</code>	Contenedor del panel embebido: App Bridge, Polaris y el menú lateral.
<code>/app/_index</code>	Panel de inicio con los contadores y el aviso legal.
<code>/app/requests</code> · <code>/app/requests/:id</code>	Listado y ficha de solicitud.
<code>/app/settings/{general,deadline,exclusions,b2b,emails}</code>	Las cinco pestañas de configuración.
<code>/app/billing</code> · <code>/app/billing/callback</code>	Planes y vuelta de la aprobación.
<code>/auth/*</code> · <code>/auth/login</code>	OAuth y acceso manual.
<code>/apps/desistimiento</code> , <code>/proxy/*</code> , <code>/count</code> , <code>/*</code>	Las cinco entradas del proxy público, todas reexportando <code>app/lib/proxy-handler.server.ts</code> .
<code>/webhooks/*</code>	Recepción única de todos los webhooks.

Webhooks

`app/uninstalled`, `app_subscriptions/update`, `customers/data_request`, `customers/redact` y `shop/redact`. Los tres últimos son de cumplimiento obligatorio del RGPD y se configuran en `shopify.app.toml`.

Ficheros de configuración y despliegue

- `shopify.app.toml` — identificador de la app, permisos, URLs de retorno, webhooks y el proxy (`/apps/desistimiento`).
- `.env.example` — plantilla de variables de entorno. **El `.env` real vive solo en el servidor.**
- `deploy/setup.sh` — primera instalación completa.
- `deploy/deploy.sh` — despliegues siguientes.
- `deploy/setup.php` — el mismo proceso desde el navegador. Requiere `WKIT_SETUP_TOKEN` y `WKIT_DB_PASS` en el entorno. **Bórralo del servidor al acabar.**
- `deploy/ecosystem.config.cjs` — PM2: proceso `desistimiento`, puerto 3001, reinicio a los 512 MB.
- `deploy/nginx-vhost.conf` — proxy inverso y bloqueo de rutas internas.
- `.ftppignore` — lo que nunca debe subirse.

Cron y permisos

No hay tareas programadas: todo funciona por webhook o por petición del usuario. En el servidor, el proceso corre bajo el usuario `shopify` y el único directorio que necesita escritura es `storage/logs/`.

Registros

`storage/logs/pm2-out.log` y `storage/logs/pm2-error.log`, escritos por PM2. Se consultan con `pm2 logs desistimiento --lines 40`. Desde esta revisión no contienen credenciales ni datos personales.